Introduction to Internet of Things

Perception Layer - Part 4

Content

1. PWM.
2. Time measurement.
3. HC-SR04
4. DHT11
5. I2C

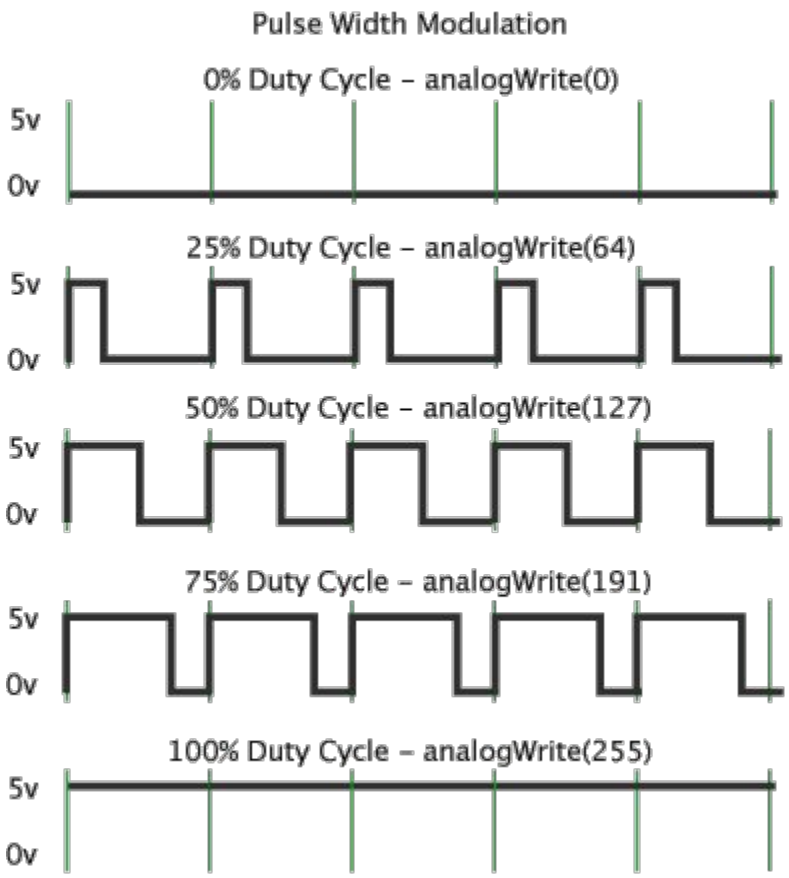
PWM

Pulse Width Modulation

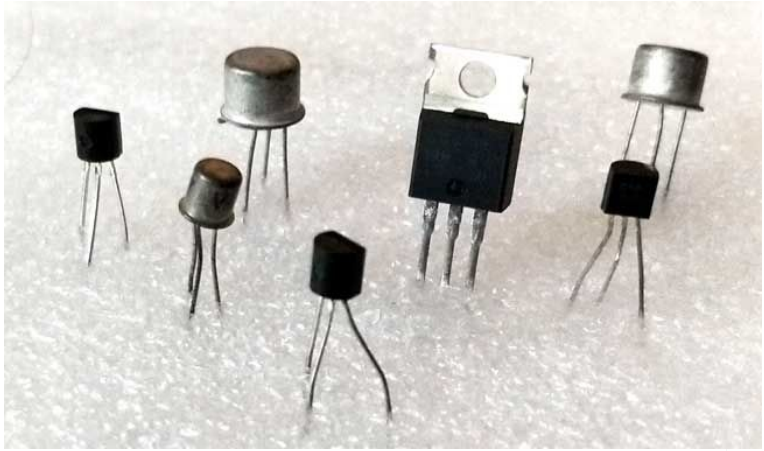
Changes a voltage/current signal with

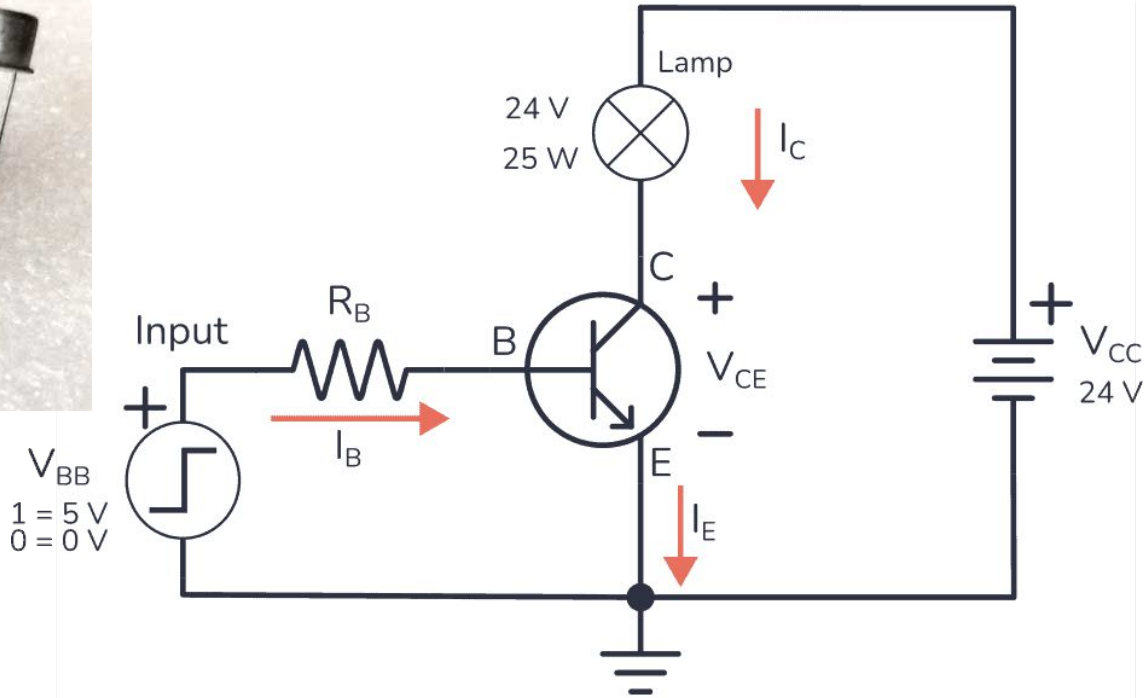
pulse time duration

An output pin of the ESP32 can do this job

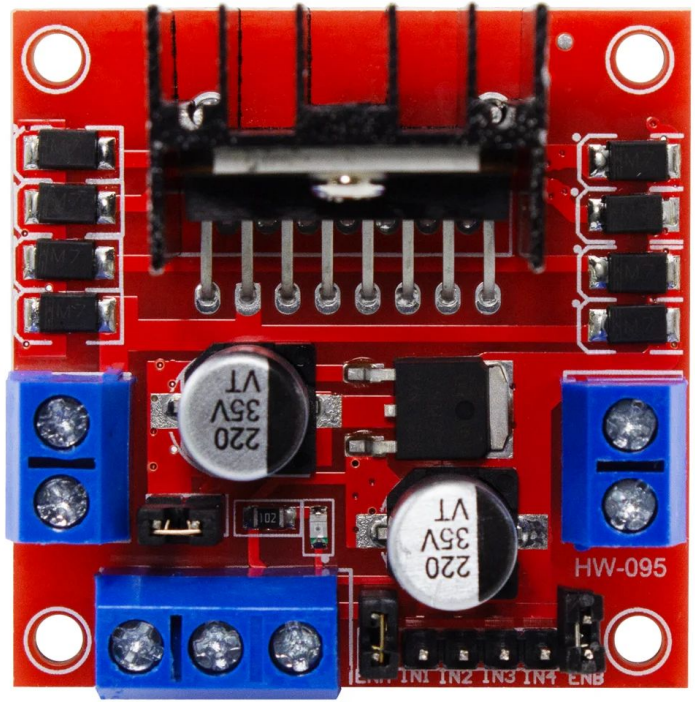


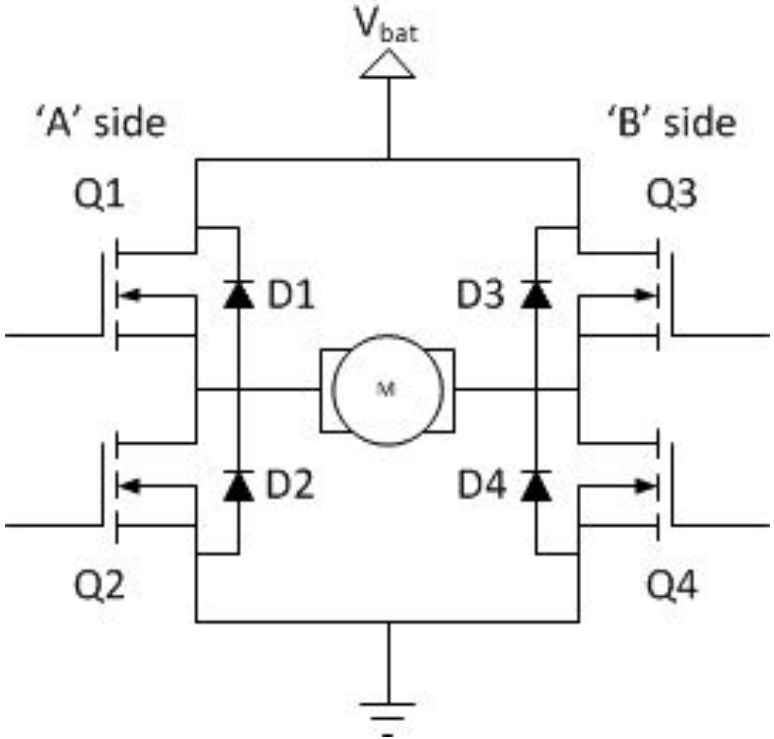
PWM





PWM


ENB IN4 IN3 IN2 IN1
220
35V
VT
220
35V
VT
HW-095


V_{bat}
'A' side
'B' side
Q1
Q3
D1
D3
M
D2
D4
Q2
Q4

PWM

Header part

```
#include <stdio.h>
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "freertos/queue.h"
#include "driver/ledc.h"
#include "esp_log.h"

#define PWM_GPIO 18
#define PWM_FREQ 5000
#define PWM_RES  LEDC_TIMER_10_BIT
#define PWM_MAX  1023
#define SPEED_MODE LEDC_LOW_SPEED_MODE // LEDC_HIGH_SPEED_MODE

#define RTOS_delay(x) vTaskDelay(pdMS_TO_TICKS(x))
#define TAG "PWM_TAG"
QueueHandle_t pwmQueue;
```

PWM

```c
void pwm_init()
{
    ledc_timer_config_t timer = {
        .speed_mode       = SPEED_MODE,
        .timer_num        = LEDC_TIMER_0,
        .duty_resolution  = PWM_RES,
        .freq_hz          = PWM_FREQ,
        .clk_cfg          = LEDC_AUTO_CLK
    };
    ledc_timer_config(&timer);
    ledc_channel_config_t channel = {
        .gpio_num   = PWM_GPIO,
        .speed_mode = SPEED_MODE,
        .channel    = LEDC_CHANNEL_0,
        .timer_sel  = LEDC_TIMER_0,
        .duty       = 0,
        .hpoint     = 0
    };
    ledc_channel_config(&channel);
}
```

```c
void pwm_task(void *pvParameters)
{
    int duty;
    while(1)
    {
        if(xQueueReceive(pwmQueue, &duty, portMAX_DELAY))
        {
            ledc_set_duty(SPEED_MODE, LEDC_CHANNEL_0, duty);
            ledc_update_duty(SPEED_MODE, LEDC_CHANNEL_0);
            ESP_LOGI(TAG, "Duty recibido: %d", duty);
        }
    }
}
```

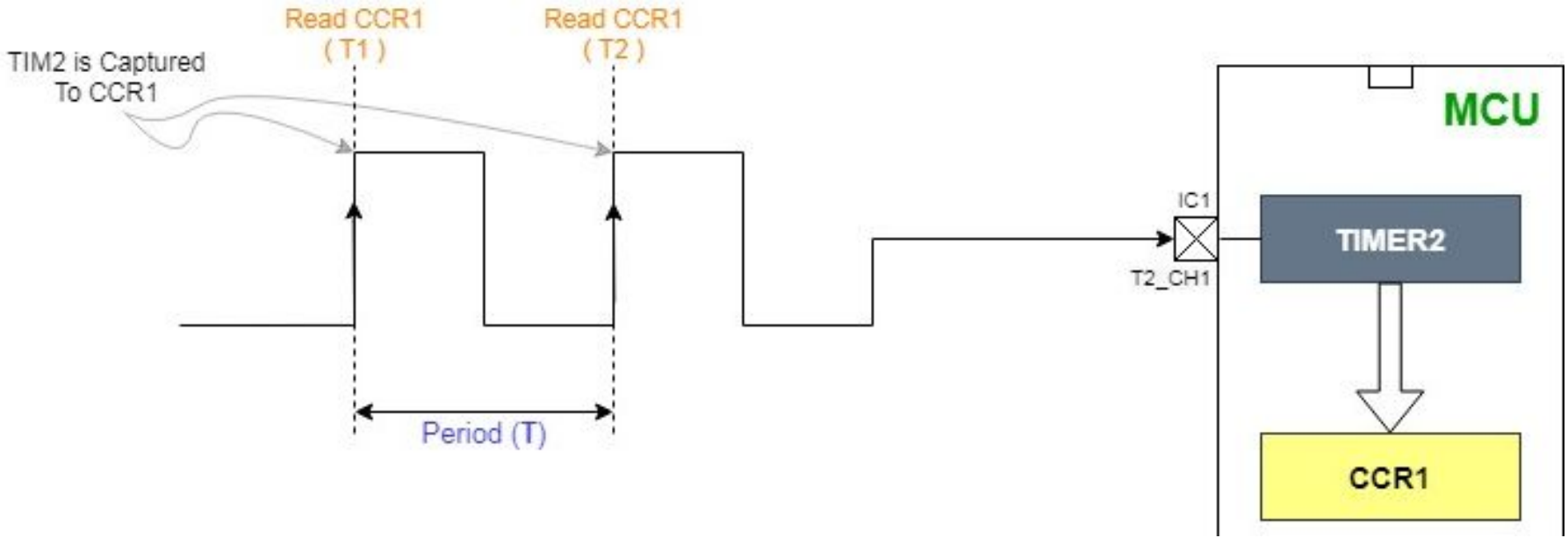
PWM

```c
void generator_task(void *pvParameters)
{
    int duty = 0;
    while(1)
    {
        xQueueSend(pwmQueue, &duty, portMAX_DELAY);
        ESP_LOGI(TAG, "Duty enviado: %d", duty);
        duty += 50;
        if(duty > PWM_MAX)
            duty = 0;
        RTOS_delay(500);
    }
}
```

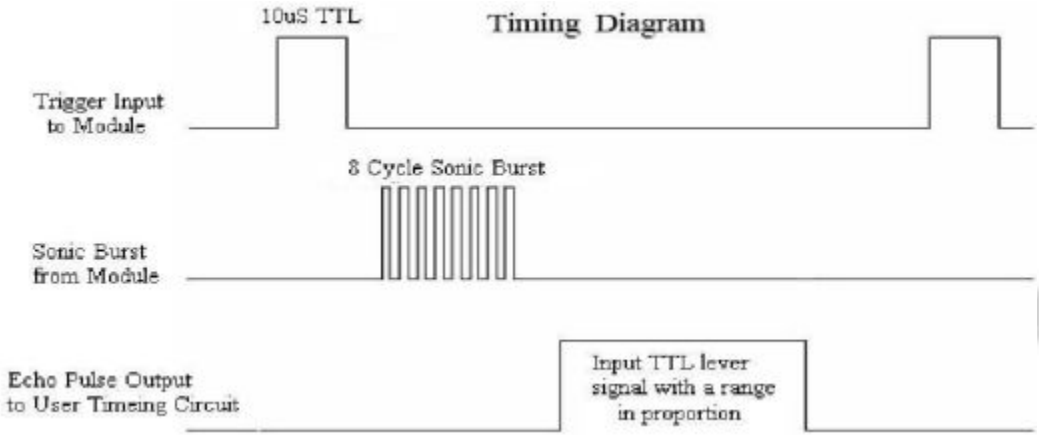
```c
void app_main()
{
    pwm_init();
    pwmQueue = xQueueCreate(5, sizeof(int));
    xTaskCreate(pwm_task, "PWM_Task", 2048, NULL, 5, NULL);
    xTaskCreate(generator_task, "Generator_Task", 2048, NULL, 4, NULL);
}
```

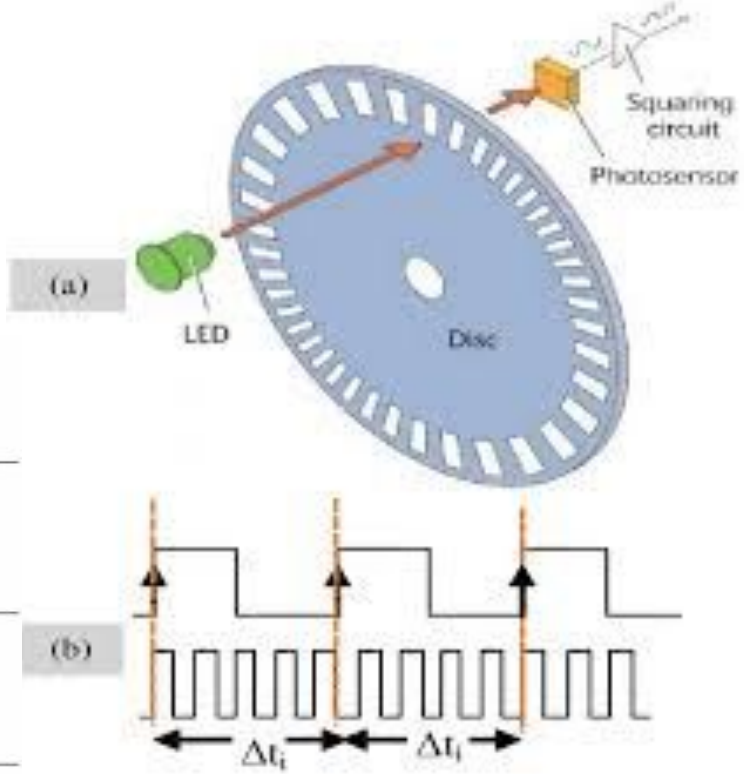
Time measurement

Time measurement



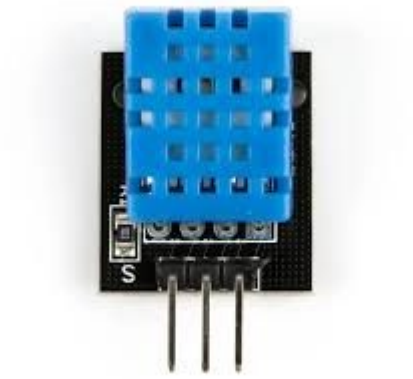
Time measurement





DHT11

Datasheet



El DHT11 es un sensor digital de bajo costo para medir temperatura y humedad relativa

Utiliza una interfaz serial de un solo cable (one-wire) para enviar datos calibrados (alta exigencia de timing)

Se debe actualizar la lectura de datos cada 1-2 segundos.

Incluye un sensor capacitivo de humedad y un termistor NTC para temperatura.

Señal de Salida: Digital.

Voltaje de Operación: 3.3V a 5V DC.

Precisión y Rango:

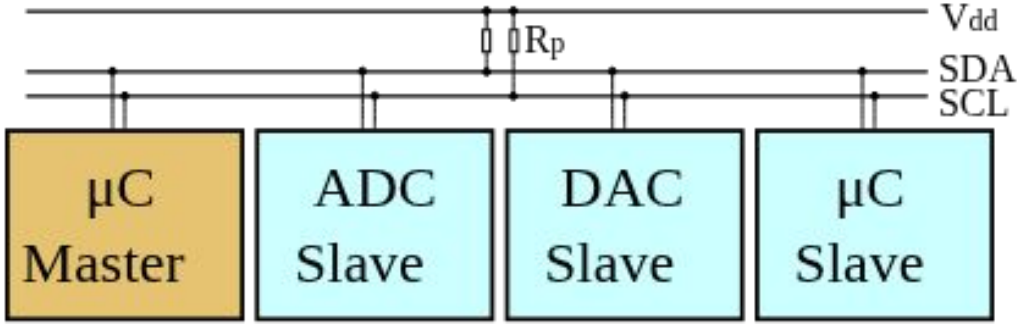
Humedad: 20-90%.

Temperatura: 0 a 50°C

IIC

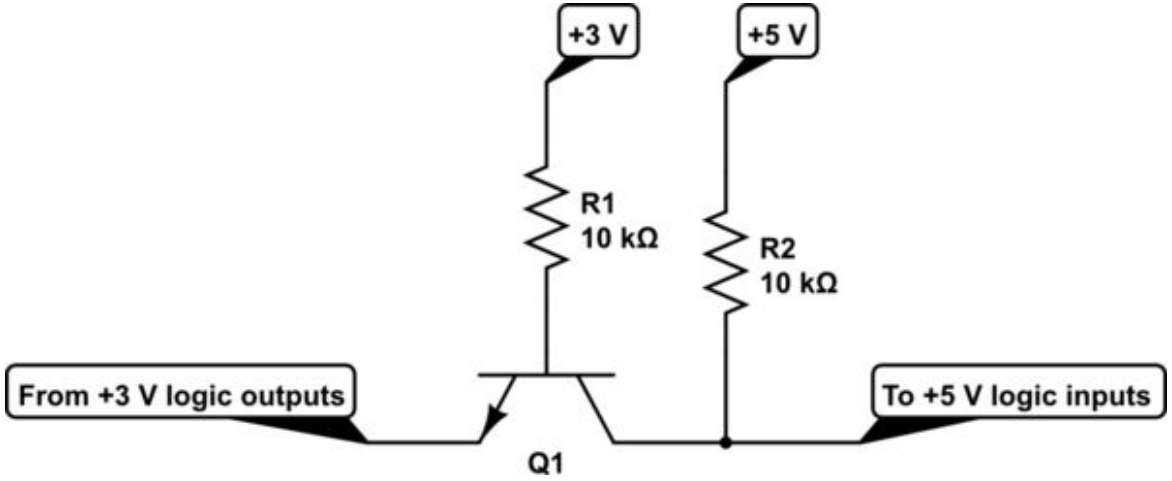
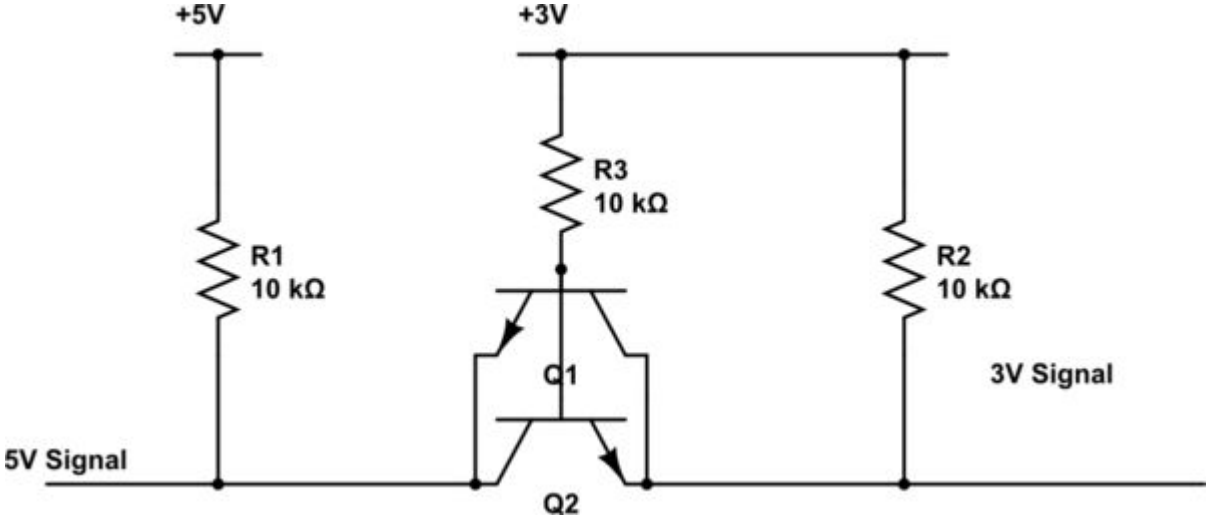
Interface de comunicación síncrona

- Sensores: Acelerómetros, IMUs
- Pantallas
- Real Time Clocks



IIC

- Solo dos hilos: Usa una línea de datos (SDA) y una de reloj (SCL), ambas requieren resistencias pull-up.
- Direccionamiento: El maestro envía la dirección del esclavo (7 bits) + bit de lectura/escritura (1 bit) para iniciar la comunicación.
- Confirmación (ACK/NACK): Cada byte transmitido es seguido por un bit de reconocimiento (ACK) del receptor, lo que confirma la recepción.
- Velocidad: Las velocidades estándar son 100 kbit/s (modo estándar), 400 kbit/s (modo rápido) y hasta 3.4 Mbit/s (modo alta velocidad).
- Multimaestro: Soporta múltiples maestros y esclavos en el mismo bus.
- Fin de comunicación: Se indica mediante una transición de bajo a alto en SDA mientras SCL está alto (condición de STOP).
- Ventajas del I2C:
- Requiere pocos pines del microcontrolador.
- Simplifica el cableado al permitir conectar hasta 127/128 dispositivos en el mismo bus.
- Es robusto para comunicación entre chips en la misma placa

IIC

Discrete level converters